



Módulo 3

IaC Core — Servicios GCP

Curso: Terraform Hands-On (GCP)

4 h (240 min)

Objetivos de Aprendizaje

- Configurar el provider `hashicorp/google` con ADC y declarar *zona*, *región* y *proyecto* como variables reutilizables.
- Distinguir entre **recursos** y **data sources** del provider Google y aplicar `lifecycle` para atributos `ForceNew`.
- Diseñar una **VPC en modo custom** con subnets regionales segmentadas por tier (app, middleware, lockers, datos) y reglas de firewall basadas en tags.
- Configurar **Cloud Router + Cloud NAT** para que las VMs en subnets privadas tengan salida a Internet sin IP pública.
- Crear **instance templates** y **Managed Instance Groups (MIG)** con health checks, autohealing y rolling updates.
- Desplegar **Cloud SQL for PostgreSQL** con alta disponibilidad regional, IP privada y SSL, consumiendo el módulo `cloudsql` del M2.
- Aplicar **labels obligatorios** a todos los recursos para reportar coste (`cost-center` , `environment` , `managed-by`).
- Validar el despliegue end-to-end con `terraform plan` limpio, `terraform output` y comandos `gcloud` de inspección.

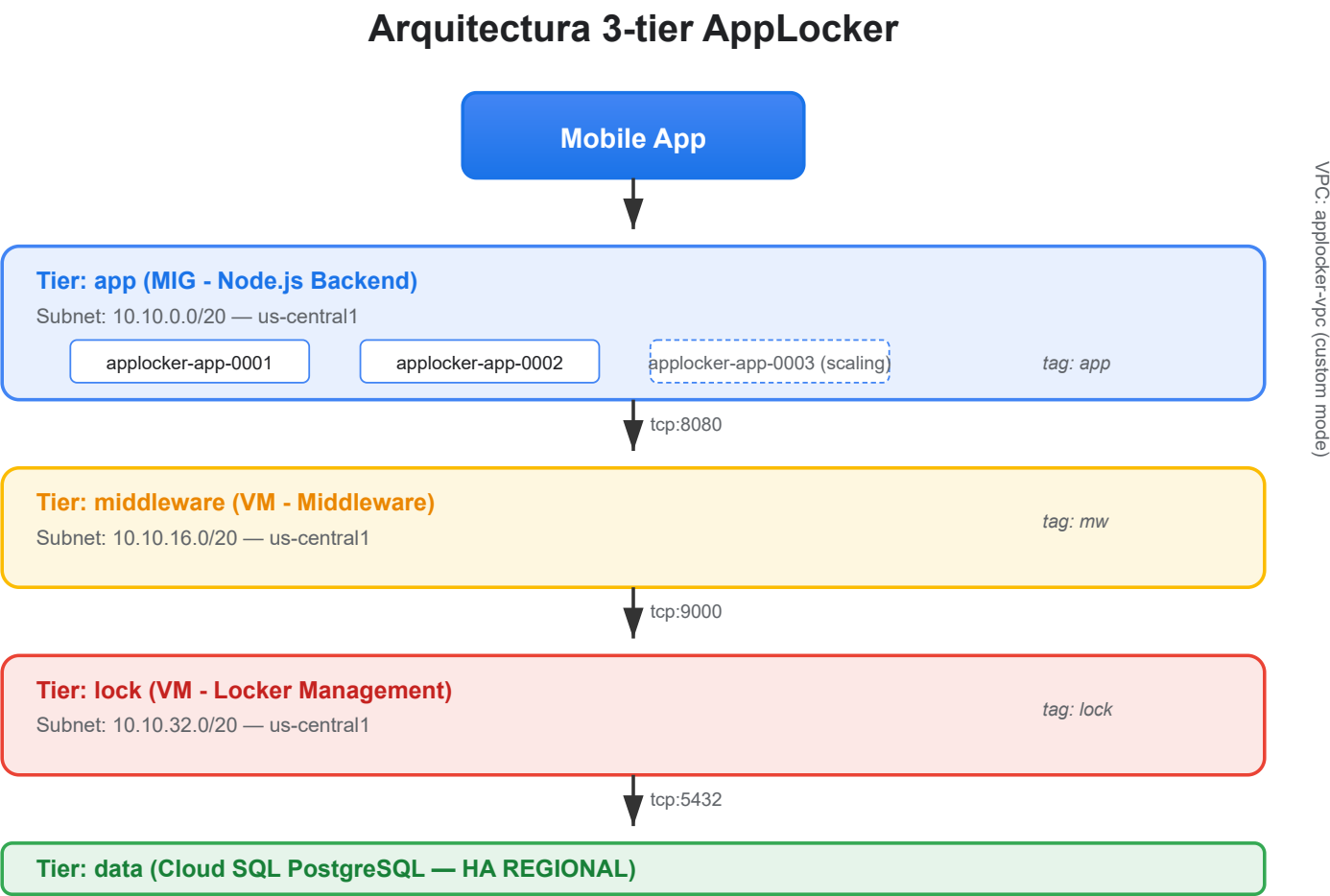
0. Apertura y objetivos

El core de la infraestructura productiva

0.1 Recap de M1 y M2

- **M1:** bucket `applocker-tf-state-<sufijo>` creado, versionado y bajo control de Terraform con backend GCS.
- **M2:** módulo `cloudsql` publicado en el private registry (GCS) y versionado en `v1.0.0`.
- En este módulo **no** se introduce state nuevo, **no** se crean módulos nuevos y **no** se tocan pipelines. Es 100% servicios GCP core.

0.2 Caso AppLocker — arquitectura objetivo



Nota técnica: el caso original menciona "MongoDB" pero **Cloud SQL no soporta MongoDB**. Tratamos el datastore como Cloud SQL for PostgreSQL.

0.3 Distribución del tiempo

- Apertura y objetivos: **5 min** (2%)
- Bloque teórico: **55 min** (23%) — repartido en 4 secciones
- Lab guiado: **165 min** (69%) — el grueso del módulo
- Cierre, Q&A y preparación de M4: **15 min** (6%)

1. Teoría — Google Provider en profundidad

≈ 14 min

1.1 Inicialización y autenticación (ADC)

- **Application Default Credentials (ADC):** Terraform lee `~/.config/gcloud/application_default_credentials.json`.
- **Alternativas:** variable `GOOGLE_CREDENTIALS` con el JSON, o service account adjunta al runner (en CI).
- `gcloud auth application-default login` deja las credenciales listas para Terraform.

Tip: en CI, preferible adjuntar la SA al runner que pegar JSON en secretos.

1.2 Configuración recomendada del provider

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    google = {  
      source  = "hashicorp/google"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "google" {  
  project = var.project_id  
  region  = var.region  
  zone    = var.zone  
  
  default_labels = {  
    environment = terraform.workspace  
    managed-by  = "terraform"  
    cost-center = "media-market-platform"  
  }  
}
```

`default_labels` aplica un set obligatorio de labels a todo recurso que los soporte — ideal para reporte de coste (FinOps).

1.3 Recursos vs data sources

- **Recursos** (`resource "google_*" "..."`): crean, leen, actualizan y destruyen infraestructura real.

Data sources (`data "google_*" "..."`): solo lectura. Sirven para:

- último image de una familia (`data "google_compute_image"`)
- outputs de otro state (`data "terraform_remote_state"`)
- listar zonas disponibles (`data "google_compute_zones"`)

```
data "google_compute_zones" "available" {}

resource "google_compute_instance" "vm" {
  name          = "applocker-app-${count.index}"
  zone          = data.google_compute_zones.available.names[0]
  machine_type  = "e2-medium"
  # ...
}
```

1.4 Atributos `ForceNew` y `lifecycle`

- **ForceNew:** cambiar su valor obliga a destruir y recrear el recurso. Ejemplo: `region` en `google_sql_database_instance`.

`lifecycle` permite control fino:

- `prevent_destroy = true` — protege contra `destroy` accidental
- `create_before_destroy = true` — zero downtime
- `ignore_changes = [labels["x"]]` — atributos gestionados fuera de TF

Anti-patrones: `ignore_changes` en todo el recurso enmascara drift real. `prevent_destroy = false` en recursos críticos de prod es una bomba de relojería.

 [Ref. oficial — Google Provider](#)

2. Teoría — Compute Engine y Managed Instance Groups

≈ 14 min

2.1 Instancias: tipos, imágenes y discos

Tipos de máquina (familia · tamaño):

- `e2-medium`, `e2-standard-2` — uso general (dev/staging)
 - `n2-standard-4` — producción
 - `c2-standard-8` — workloads CPU-bound
 - `m1-ultramem-96` — in-memory DB / Spark
- **Imágenes recomendadas para AppLocker:** `cos-cloud/cos-stable` (Container-Optimized OS) — mínimo, seguro por defecto.
 - **Discos:** boot persistente (`pd-balanced`), datos `pd-ssd` para DB local.

2.2 Instancia mínima — VM sin IP pública

```
resource "google_compute_instance" "locker_mgmt" {
  name          = "applocker-locker-mgmt"
  zone          = var.zone
  machine_type  = "e2-small"

  boot_disk {
    initialize_params {
      image = "cos-cloud/cos-stable"
      size  = 20
      type  = "pd-balanced"
    }
  }
}

network_interface {
  subnetwork = google_compute_subnetwork.lock.self_link
  access_config {
    # Sin IP pública: solo egress vía Cloud NAT
  }
}
```

Adelanto de M4: por ahora usamos la service account default con `cloud-platform`. En M4 crearemos SAs dedicadas con roles mínimos.

2.3 Instance templates

- Es el "plano" que usa un MIG para crear N instancias idénticas.
- **NO se puede modificar** después de creado: es *immutable*. Para cambios => nueva plantilla + rolling update.
- `create_before_destroy = true` para mantener disponibilidad.

```
resource "google_compute_instance_template" "app" {
  name_prefix = "applocker-app-tmpl-"
  machine_type = "e2-standard-2"
  region      = var.region

  disk {
    source_image = "cos-cloud/cos-stable"
    auto_delete = true
    boot        = true
  }

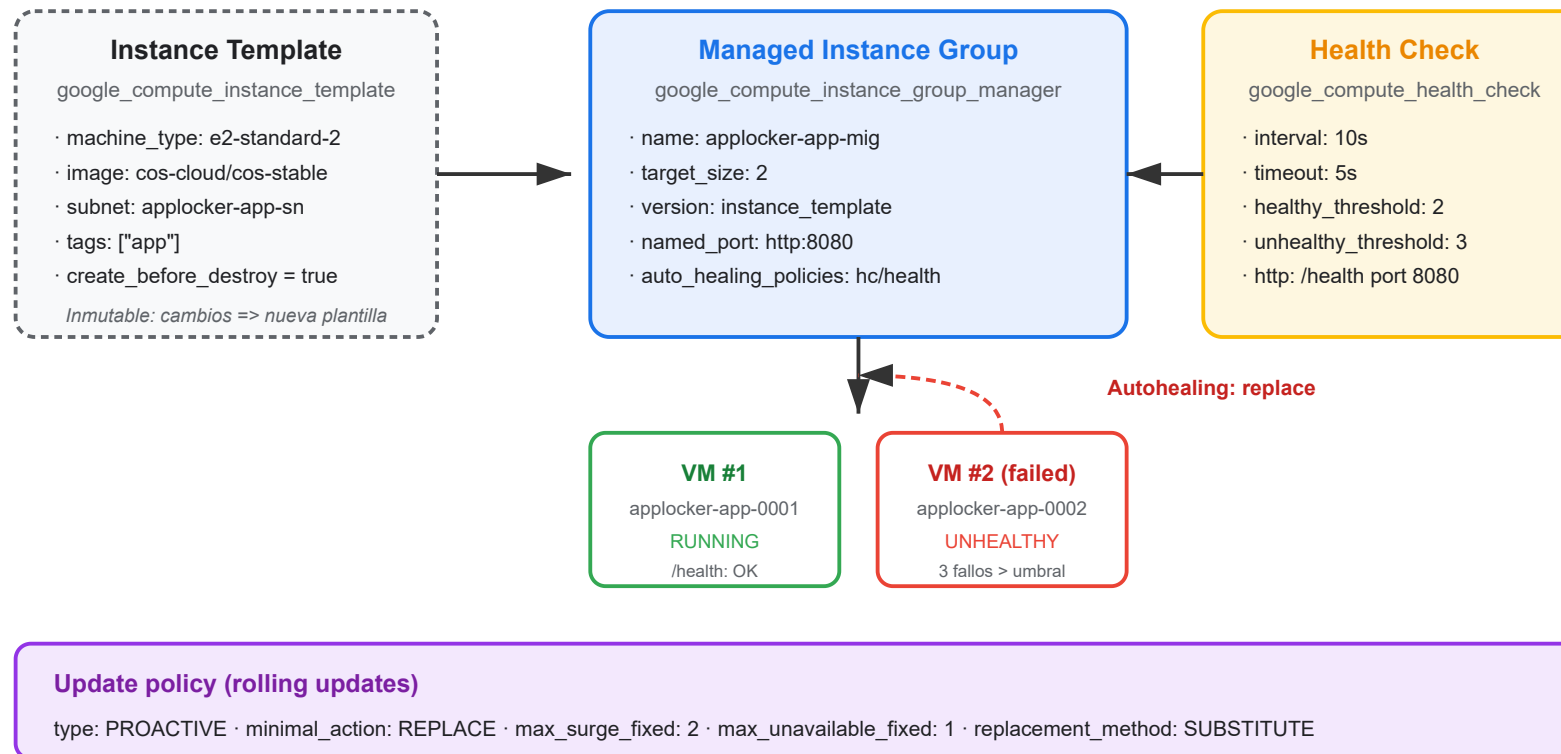
  network_interface {
    subnetwork = google_compute_subnetwork.app.self_link
  }

  tags = ["app"]
  labels = { tier = "app" }

  lifecycle {
```

2.4 MIG: stateless vs stateful

Managed Instance Group (stateless) + Autohealing



- **Stateless MIG** (caso AppLocker): VMs intercambiables. Sesión y estado fuera (Redis, DB).
- **Stateful MIG**: cada VM tiene disco persistente con identificador fijo. Útil para software legacy. *Poco recomendado en cloud-native.*

2.5 Autohealing + health checks

```
resource "google_compute_health_check" "app" {
  name = "applocker-app-hc"

  check_interval_sec = 10
  timeout_sec        = 5
  healthy_threshold  = 2
  unhealthy_threshold = 3

  http_health_check {
    port          = 8080
    request_path = "/health"
  }
}

resource "google_compute_instance_group_manager" "app_mig" {
  # ...
  auto_healing_policies {
    health_check      = google_compute_health_check.app.self_link
    initial_delay_sec = 60
  }
}
```

Si una instancia falla `unhealthy_threshold` veces consecutivas, el MIG la reemplaza automáticamente.

2.6 Rolling updates

- El MIG reemplaza instancias gradualmente: `max_surge_fixed` nuevas VMs antes de tumbar las viejas.
- **Max unavailable:** cuántas VMs pueden estar caídas a la vez. Para backend con $N \geq 2$ suele ser `1`.

```
update_policy {  
  type           = "PROACTIVE"  
  minimal_action = "REPLACE"  
  max_surge_fixed = 2  
  max_unavailable_fixed = 1  
  replacement_method = "SUBSTITUTE"  
}
```

 [Ref. oficial — Instance Groups](#)

3. Teoría — Cloud SQL con Terraform

≈ 14 min

3.1 Ediciones, tiers y HA

- **Ediciones:** Enterprise (default) · Enterprise Plus (HA reforzada, read pools, SLA mayor).

Tiers:

- `db-f1-micro`, `db-g1-small` — dev/sandbox
- `db-custom-2-7680` — 2 vCPU + 7.5 GB RAM (producción pequeña)
- `db-custom-8-30720` — 8 vCPU + 30 GB RAM (producción media)

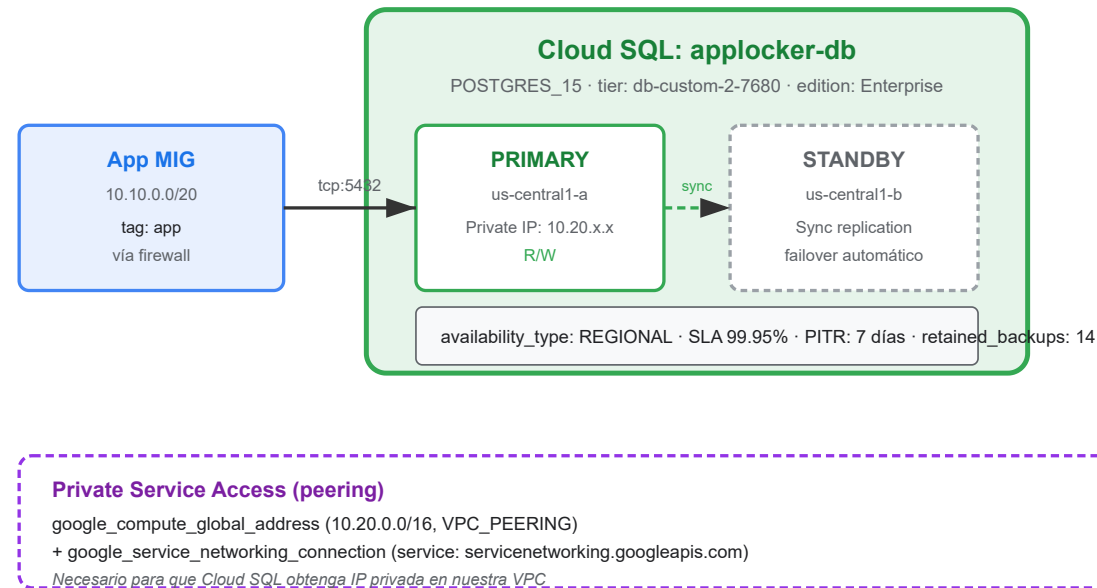
HA (`availability_type`):

- `ZONAL` — sin SLA de HA
- `REGIONAL` — failover automático a otra zona. SLA 99.95%. **Recomendado para prod.**

- **Disco:** `PD_SSD` (default) o `PD_HDD`. Autoresize habilitado hasta `disk_autoresize_limit`.

3.2 Conexión privada + SSL obligatorio

Cloud SQL for PostgreSQL — HA REGIONAL + Private IP



- **IP privada obligatoria en prod:** `ipv4_enabled = false` + `private_network`.
- **Service Networking:** peering automático entre la VPC y `services.googleapis.com`. Necesita:
 - API `servicenetworking.googleapis.com` habilitada
 - `google_compute_global_address` con `purpose = "VPC_PEERING"`
- **SSL obligatorio en tránsito:** `require_ssl = true`.

3.3 Backups y PITR

```
backup_configuration {  
  enabled           = true  
  start_time        = "03:00"  
  location          = "eu"  
  point_in_time_recovery_enabled = true  
  transaction_log_retention_days = 7  
  retained_backups  = 14  
}
```

- **PITR (Point In Time Recovery):** restaurar a cualquier segundo de los últimos 7 días.
- Crítico para incidentes tipo *"DROP TABLE accidental"*.

Alternativa: Cloud SQL Proxy cifra la conexión sin abrir firewall. Útil para apps fuera de GCP. En este curso no lo instalamos (las VMs están en la misma VPC).

 [Ref. oficial — Cloud SQL best practices](#)

4. Teoría — VPC, subnets y firewall

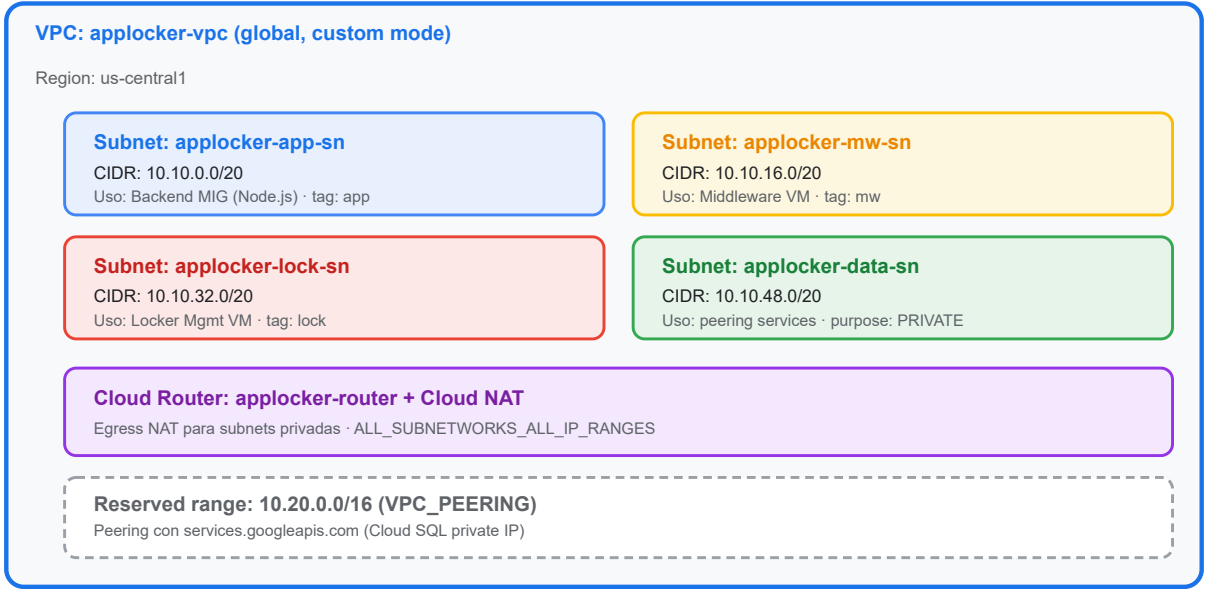
≈ 13 min

4.1 VPC global vs subnets regionales

- Una **VPC en GCP es global**, pero las **subnets son regionales**.
- Un recurso en `us-central1` puede hablar con otro en `europa-west4` sin salir de la red privada de GCP.
- **Modo automático**: GCP elige los CIDR. *No recomendado en prod.*
- **Modo custom**: tú defines cada CIDR. **Recomendado para AppLocker.**

4.2 CIDR planning — 4 subnets segmentadas por tier

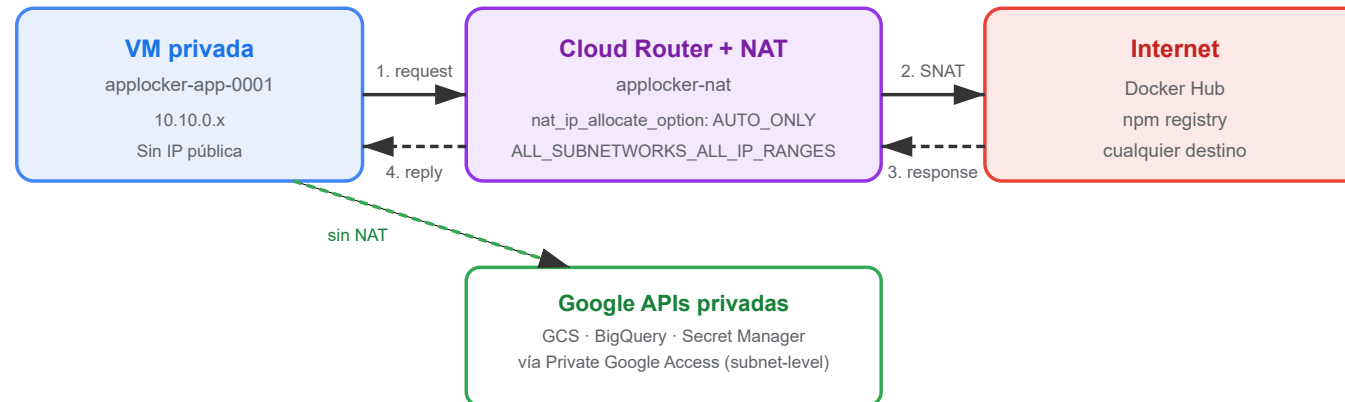
VPC custom mode — Segmentación por tiers



Subnet	CIDR	Uso
app	10.10.0.0/20	Backend MIG (Node.js)
middleware	10.10.16.0/20	Middleware VM
lock	10.10.32.0/20	Locker Management VM
data	10.10.48.0/20	Reserved para Cloud SQL (PRIVATE)

4.3 Private Google Access + Cloud NAT

Egress sin IP pública: Cloud Router + Cloud NAT



Diferencia clave: Private Google Access (subnet) usa la red privada de Google. Cloud NAT requiere Cloud Router y permite salida a Internet general. Si solo consumes GCS/BigQuery, Private Google Access basta. Para Docker Hub u otros, necesitas Cloud NAT.

- **Private Google Access** (subnet-level): VMs sin IP pública llegan a las APIs de Google (GCS, BigQuery, Secret Manager) por red privada.
- **Cloud Router + Cloud NAT:** egress a Internet general (Docker Hub, npm) sin asignar IP pública.

Si olvidas `private_ip_google_access = true`, las VMs quedan aisladas de las APIs de Google. Es uno de los errores más comunes.

4.4 Firewall por tags — zero-trust a nivel red

Firewall por tags — Zero-trust a nivel red

 **Implicit deny**

Todo lo no permitido está bloqueado por defecto

applocker-app-to-mw
INGRESS · source_tags: [app] · target_tags: [mw] · tcp:8080 · allow

applocker-mw-to-lock
INGRESS · source_tags: [mw] · target_tags: [lock] · tcp:9000 · allow

applocker-lock-to-data
INGRESS · source_tags: [lock] · target_tags: [data] · tcp:5432 · allow

applocker-ssh-iap
INGRESS · source_ranges: [35.235.240.0/20] (IAP) · target_tags: [app, mw, lock] · tcp:22 · allow

applocker-egress-https
EGRESS · source_tags: [app, mw, lock] · tcp:443 · allow (salida vía Cloud NAT)

 **Regla de oro**

Cada VM lleva tags (app, mw, lock, data). Las reglas se aplican POR TAG, nunca por IP directa.

Forma de la regla: source_tags + target_tags + allow { protocol + ports } + log_config

Regla de oro: las reglas se aplican **por tag**, nunca por IP directa. Cada VM lleva tags que la identifican (`app` , `mw` , `lock` , `data`).

4.5 Ejemplo: reglas de firewall para AppLocker

```
resource "google_compute_firewall" "app_to_mw" {
  name      = "applocker-app-to-mw"
  network   = google_compute_network.applocker.id
  direction = "INGRESS"

  source_tags = ["app"]
  target_tags = ["mw"]

  allow {
    protocol = "tcp"
    ports    = ["8080"]
  }

  log_config {
    metadata = "INCLUDE_ALL_METADATA"
  }
}

resource "google_compute_firewall" "ssh_ian" {
```

Implicit deny: en GCP todo lo no permitido está bloqueado. Solo declaramos reglas `allow`.

 [Ref. oficial — VPC Firewalls](#)

5. Lab guiado — Arquitectura 3-tier de AppLocker

≈ 165 min · 100% instructor-led

5.1 Prerrequisitos

- Terraform `~> 1.5` instalado.
- `gcloud` autenticado con permisos `compute.admin`, `cloudsql.admin`, `iam.serviceAccountUser`.
- Bucket de state `applocker-tf-state-<sufijo>` existe (del M1).
- Módulo `cloudsql` `v1.0.0` publicado en `gs://applocker-tf-state-<sufijo>/modules/cloudsql/1.0.0/cloudsql.zip` (del M2).

```
gcloud services enable \  
  compute.googleapis.com \  
  sqladmin.googleapis.com \  
  servicenetworking.googleapis.com \  
  cloudresourcemanager.googleapis.com
```

5.2 Crear la VPC y subnets (20 min)

1. Estructura de archivos:

```
mkdir -p infra/live/compute infra/live/network
```

2. Crear `infra/live/network/main.tf` con la VPC y las 4 subnets del bloque 4.2.

3. Crear `infra/live/network/variables.tf` con `project_id`, `region`, `zone`.

4. Crear `infra/live/network/outputs.tf` con `vpc_self_link`, `subnet_ids` y `subnet_self_links` por tier.

5. `terraform init && terraform plan && terraform apply`.

5.3 Cloud Router + Cloud NAT (15 min)

1. Añadir a `infra/live/network/main.tf` los recursos `google_compute_router` y `google_compute_router_nat` del bloque 4.3.
2. `terraform apply` .
3. Validar:

```
gcloud compute routers nats list \  
--router=applocker-router --region=us-central1
```


5.4 Reglas de firewall segmentadas por tags (15 min)

1. Añadir a `infra/live/network/main.tf` los tres `google_compute_firewall` del bloque 4.4.
2. `terraform apply` .
3. Verificar:

```
gcloud compute firewall-rules list \  
--filter="network=applocker-vpc"
```

5.5 Plantilla de instancia para el backend (15 min)

1. Crear `infra/live/compute/main.tf` con el `google_compute_instance_template` del bloque 2.3.
2. Añadir startup script que prepare el entorno para Node.js (placeholder, se endurece en M4):

```
metadata_startup_script = <<-EOT
#!/bin/bash
docker-credential-gcr configure-docker --quiet
EOT
```

3. `terraform apply` y verificar la plantilla en `gcloud compute instance-templates list`.

5.6 Managed Instance Group del backend (15 min)

1. Añadir a `infra/live/compute/main.tf` el `google_compute_instance_group_manager` del bloque 2.4.
2. `terraform apply` .
3. Verificar las 2 VMs:

```
gcloud compute instance-groups list-instances \  
  applocker-app-mig --zone=us-central1-a
```

5.7 Health check (10 min)

1. Añadir el `google_compute_health_check` y el `auto_healing_policies` al MIG (bloque 2.5).
2. `terraform apply` .
3. **Simulación didáctica:** parar una VM con `gcloud compute instances stop` y verificar que el MIG la reemplaza en ~1 minuto.

5.8 Cloud SQL privado con HA (consumiendo el módulo del M2) — 30 min

1. API `servicenetworking` ya habilitada (5.1). Crear el rango de peering:

```
resource "google_compute_global_address" "private_ip_range" {
  name          = "applocker-private-ip-range"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  network       = google_compute_network.applocker.id
}

resource "google_service_networking_connection" "private_vpc" {
  network                 = google_compute_network.applocker.id
  service                 = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.private_ip_range.name]
}
```

1. Crear `infra/live/cloudsql/main.tf` que consume el módulo del M2:

```
module "cloudsql" {  
  source = "gcs::https://www.googleapis.com/storage/v1/applocker-tf-state-<sufijo>/modules/cloudsql/1.0.0/cloudsql.zip"  
  version = "1.0.0"  
  
  project_id      = var.project_id  
  name            = "applocker-db"  
  region          = var.region  
  tier              = "db-custom-2-7680"  
  availability_type = "REGIONAL"  
  database_version = "POSTGRES_15"  
  private_network  = module.network.vpc_self_link  
  deletion_protection = true  
  
  depends_on = [google_service_networking_connection.private_vpc]  
}
```

2. `terraform init -upgrade` (fuerza descarga del módulo desde GCS) y `terraform apply`.

3. Verificar:

```
gcloud sql instances describe applocker-db \  
--format="value(state,region,availabilityType)"
```

Si olvidas `depends_on` entre el peering y el módulo Cloud SQL: race condition. El apply fallará aleatoriamente.

5.9 Labels obligatorios (10 min)

1. Verificar que el provider tiene `default_labels` configurados (bloque 1.2).
2. `terraform plan` debe mostrar los labels aplicados a VPC, subnets, NAT, health check, instance template y Cloud SQL.
3. Si algún recurso no muestra los labels esperados, revisar `default_labels` en el provider y re-aplicar.

5.10 Validación final (15 min)

- `terraform plan` debe estar limpio: *"No changes. Your infrastructure matches the configuration."*
- Inspección con `terraform output`:

```
terraform output -json | jq .
```

Debe devolver al menos:

- `vpc_self_link`
 - `app_mig_self_link`
 - `cloudsql_connection_name`
 - `cloudsql_private_ip`
- Verificación end-to-end desde una VM del MIG hacia Cloud SQL:

```
gcloud compute ssh applocker-app-XXXX \  
  --zone=us-central1-a \  
  --command="nc -zv <CLOUDSQL_PRIVATE_IP> 5432"
```

Esperado: *"Connection to <ip> 5432 port [tcp/postgres] succeeded!"*

5.11 Limpieza y commit

Importante: NO destruir los recursos en este módulo. La arquitectura es la base para M4 (seguridad), M5 (GitOps) y M6 (migración zero-downtime).

- Solo se destruyen (si el formador confirma) las VMs individuales dentro del MIG para validar el autohealing.
- Eliminar archivos locales `*.tfstate*` y `.terraform/` antes de commit.
- Commit:

```
git add infra/  
git commit -m "feat(M3): desplegar arquitectura 3-tier de AppLocker (VPC + MIG + Cloud SQL)"  
git push origin main
```

6. Cierre, Q&A y preparación de M4

≈ 15 min

6.1 Recap del módulo

- VPC en modo custom con 4 subnets segmentadas por tier.
- Cloud Router + Cloud NAT para egress sin IP pública.
- Firewall por tags: zero-trust a nivel red, sin reglas por IP.
- Instance template + MIG + health check + autohealing + rolling updates.
- Cloud SQL for PostgreSQL privado con HA regional consumido desde el módulo del M2.
- Labels obligatorios aplicados vía `default_labels` del provider.

6.2 Errores comunes en cohortes previas

- Olvidar `private_ip_google_access = true` y bloquear la salida a las APIs de Google.
- Crear la subnet `data` sin `purpose = "PRIVATE"` y colisionar el peering con `services`.
- Definir `availability_type = "ZONAL"` por error en prod (pierde el SLA de HA).
- Olvidar `depends_on` entre el peering y el módulo Cloud SQL → race condition.
- Firewall demasiado permisivo (`source_ranges = ["0.0.0.0/0"]`) para SSH — debe ir por IAP.

6.3 Pregunta abierta

"¿Qué pasa si la red interna del tier `app` necesita hablar con un servicio on-premises de MediaMarkt? ¿Qué tendríamos que añadir?"

Anticipo: Cloud Interconnect o Cloud VPN — fuera del alcance de este curso.

6.4 Preparación de M4

En M4 endureceremos todo esto con:

- **Service accounts dedicadas** por VM (en lugar de la default).
- **Secret Manager** para credenciales de la base de datos.
- **Roles IAM mínimos** por service account.
- **Labels de coste** adicionales (`cost-center` , `business-unit`).

Resumen

Puntos Clave

- El provider `hashicorp/google` se autentica con **ADC** y `default_labels` aplica labels obligatorios a todos los recursos.
- Una **VPC en GCP es global**; las subnets son regionales. El modo **custom** te da control total sobre CIDR.
- **Cloud Router + Cloud NAT** permiten egress a Internet sin IP pública. **Private Google Access** (subnet-level) cubre las APIs de Google.
- **Firewall por tags** = zero-trust a nivel red. Las reglas se aplican por tag, no por IP.
- Un **MIG** consume un instance template (inmutable) y aplica health checks para autohealing + rolling updates.
- **Cloud SQL** en modo `REGIONAL` ofrece HA con SLA 99.95%. IP privada + SSL obligatorio en tránsito.
- **Service Networking** (peering) es prerequisite para IP privada de Cloud SQL: rango reservado + `google_service_networking_connection`.
- Siempre valida con `terraform plan` limpio, `terraform output` y comandos `gcloud` de inspección.

Referencias

Provider y recursos Terraform

- [Google Provider — Documentación oficial](#)
- [google_compute_network](#)
- [google_compute_subnetwork](#)
- [google_compute_firewall](#)
- [google_compute_instance_template](#)
- [google_compute_instance_group_manager](#)
- [google_compute_health_check](#)
- [google_sql_database_instance](#)
- [google_compute_router_nat](#)

Documentación GCP

- [Compute Engine — Instance Groups](#)
- [Managed Instance Groups — Creating groups](#)
- [Cloud SQL for PostgreSQL — Best practices](#)
- [Cloud SQL — Alta disponibilidad](#)
- [VPC — Reference architecture](#)
- [VPC — Firewall rules](#)
- [Cloud NAT — Overview](#)
- [Private Google Access](#)
- [Service Networking_\(peering\)](#)
- [Identity-Aware Proxy_\(IAP\)](#)

Anexos internos del curso

- `anexo/gc-in-a-nutshell.md` — secciones 2 (VPC), 3 (Compute), 4 (Cloud SQL) y 5 (IAM básico).
- `anexo/terraform-in-a-nutshell.md` — sección 4 (provider lifecycle).



¡Gracias!

Módulo 3 Completado

Siguiente: Módulo 4 — Seguridad, IAM y Secret Manager